

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Algoritma BFS (Breadth First Search) dan DFS (Depth First Search)”

DISUSUN OLEH:

NOFRI ILHAM

2511531013

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

M Galid A



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum yang berjudul "Algoritma BFS (Breadth First Search) dan DFS (Depth First Search)" ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu tugas praktikum mata kuliah Struktur Data. Melalui praktikum ini, penulis mempelajari konsep dasar algoritma penelusuran graf, yaitu Breadth First Search (BFS) dan Depth First Search (DFS), serta mengimplementasikannya menggunakan bahasa pemrograman Java. Dengan adanya praktikum ini, diharapkan penulis dapat memahami cara kerja kedua algoritma tersebut, mengetahui perbedaan di antara keduanya, serta mampu menerapkannya dalam proses penelusuran data pada struktur graf.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan dan jauh dari kata sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan ini di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca mengenai algoritma BFS dan DFS.

Wassalamu'alaikum Warahmatullahi Wabarakatuh

Padang, 12 Juli 2026

Nofri Ilham

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	2
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Pengertian BFS.....	3
2.2 Pengertian DFS	3
2.3 Implementasi BFS dan DFS pada Praktikum.....	4
2.3.1 Kode Pertama(Node_2511531013).....	4
2.3.2 Kode Kedua(BTree_2511531013)	6
2.3.3 Kode Ketiga(BtreeDriver_2511531013).....	8
2.3.4 Kode Keempat(GraphTraversal_2511531013).....	10
BAB III PENUTUP	13
3.1 Kesimpulan.....	13
3.2 Saran	13
DAFTAR PUSTAKA	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Graf merupakan salah satu struktur data yang digunakan untuk merepresentasikan hubungan antar objek dalam berbagai bidang, seperti jaringan komputer, media sosial, sistem navigasi, dan pencarian rute. Dalam proses penelusuran graf, diperlukan algoritma yang mampu mengunjungi setiap simpul secara sistematis sehingga informasi yang diinginkan dapat ditemukan dengan efisien. Dua algoritma penelusuran graf yang paling umum digunakan adalah Breadth First Search (BFS) dan Depth First Search (DFS).

Breadth First Search (BFS) melakukan penelusuran secara melebar dengan mengunjungi seluruh simpul yang bertetangga terlebih dahulu sebelum berpindah ke tingkat berikutnya. Sementara itu, Depth First Search (DFS) melakukan penelusuran secara mendalam dengan mengikuti satu jalur hingga mencapai simpul terakhir sebelum kembali ke simpul sebelumnya. Kedua algoritma tersebut memiliki karakteristik dan cara kerja yang berbeda sehingga penggunaannya disesuaikan dengan kebutuhan penyelesaian suatu permasalahan.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar algoritma BFS dan DFS serta mampu mengimplementasikannya menggunakan bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan untuk membandingkan proses penelusuran yang dihasilkan oleh kedua algoritma sehingga mahasiswa dapat mengetahui kelebihan dan karakteristik masing-masing algoritma dalam penelusuran graf.

1.2 Tujuan

Adapun tujuan dari praktikum struktur data dengan materi Algoritma *BFS dan DFS* adalah sebagai berikut:

1. Memahami konsep dasar algoritma Breadth First Search (BFS) dan Depth First Search (DFS).
2. Mengimplementasikan algoritma BFS dan DFS menggunakan bahasa pemrograman Java.
3. Mengetahui proses penelusuran simpul pada struktur data graf menggunakan algoritma BFS dan DFS.
4. Membandingkan hasil penelusuran yang dihasilkan oleh algoritma BFS dan DFS.
5. Melatih kemampuan mahasiswa dalam menerapkan algoritma penelusuran graf untuk menyelesaikan suatu permasalahan.

1.3 Manfaat

Adapun manfaat dari praktikum struktur data dengan materi *BFS dan DFS* adalah sebagai berikut:

1. Menambah pemahaman mengenai konsep dan cara kerja algoritma Breadth First Search (BFS) dan Depth First Search (DFS).
2. Meningkatkan kemampuan dalam mengimplementasikan algoritma penelusuran graf menggunakan bahasa pemrograman Java.
3. Memberikan pengalaman dalam menganalisis proses penelusuran simpul pada struktur data graf.
4. Membantu memahami perbedaan karakteristik serta penerapan algoritma BFS dan DFS pada berbagai permasalahan.
5. Menjadi dasar dalam mempelajari algoritma graf yang lebih kompleks pada materi selanjutnya.

BAB II

PEMBAHASAN

2.1 Pengertian BFS

BFS, kependekan dari Breadth-first Search adalah algoritma yang digunakan untuk membuat grafik data dan mencari atau melintasi struktur tree/graph.

Algoritma ini secara efisien mengunjungi dan menandai semua node kunci dalam graph dengan cara yang akurat.

Algoritma BFS memilih satu node (titik awal atau sumber) dalam graph, kemudian mengunjungi semua node yang berdekatan dengan node yang dipilih.

Setelah algoritma mengunjungi dan menandai node awal, algoritma akan bergerak menuju node terdekat yang belum dikunjungi dan menganalisisnya.

Setelah dikunjungi, semua node ditandai. Iterasi ini berlanjut hingga semua simpul pada graph berhasil dikunjungi dan ditandai.

2.2 Pengertian DFS

DFS atau Depth-first Search adalah algoritma untuk menemukan atau melintasi sebuah graph maupun tree dengan arah lintasan secara dalam.

Eksekusi algoritma dimulai dari simpul akar dan menjelajahi setiap cabang sebelum proses backtracking (mundur menjelajahi simpul lain).

Algoritma ini menggunakan struktur data stack untuk mengingat, mendapatkan simpul berikutnya, dan untuk memulai pencarian setiap kali jalan buntu muncul di setiap iterasi.

2.3 Implementasi BFS dan DFS pada Praktikum

2.3.1 Kode Pertama(Node_2511531013)

```
1 package Pekan9_2511531013;
2
3 public class Node_2511531013 {
4     int data_1013;
5     Node_2511531013 left_1013;
6     Node_2511531013 right_1013;
7
8     public Node_2511531013(int data_1013) {
9         this.data_1013 = data_1013;
10        left_1013 = null;
11        right_1013 = null;
12    }
13
14    public void setleft_1013(Node_2511531013 node) {
15        if (left_1013 == null)
16            left_1013 = node;
17    }
18
19    public void setright_1013(Node_2511531013 node) {
20        if (right_1013 == null)
21            right_1013 = node;
22    }
23
24    public Node_2511531013 getleft_1013() {
25        return left_1013;
26    }
27
28    public Node_2511531013 getright_1013() {
29        return right_1013;
30    }
31
32    public int getdata_1013() {
33        return data_1013;
34    }
35
36    public void setdata_1013(int data_1013) {
37        this.data_1013 = data_1013;
38    }
39 }
```

```

39
40 void printPreorder(Node_2511531013 node) {
41     if (node == null)
42         return;
43     System.out.print(node.data_1013 + " ");
44     printPreorder(node.left_1013);
45     printPreorder(node.right_1013);
46 }
47
48 void printPostorder(Node_2511531013 node) {
49     if (node == null)
50         return;
51     printPostorder(node.left_1013);
52     printPostorder(node.right_1013);
53     System.out.print(node.data_1013 + " ");
54 }
55
56 void printInorder(Node_2511531013 node) {
57     if (node == null)
58         return;
59     printInorder(node.left_1013);
60     System.out.print(node.data_1013 + " ");
61     printInorder(node.right_1013);
62 }
63
64 public String print() {
65     return this.print("", true, "");
66 }
67
68 public String print(String prefix, boolean isTail, String sb) {
69     if (right_1013 != null) {
70         right_1013.print(prefix + (isTail ? "| " : " "), false, sb);
71     }
72     System.out.println(prefix + (isTail ? "\\-- " : "/-- ") + data_1013);
73     if (left_1013 != null) {
74         left_1013.print(prefix + (isTail ? " " : "| "), true, sb);
75     }
76     return sb;
77 }
78 }

```

Kode 2.1

Kode di atas merupakan kelas `Node_2511531013` yang digunakan untuk merepresentasikan setiap simpul (node) pada struktur data binary tree. Setiap node memiliki atribut `data_1013` sebagai nilai yang disimpan, serta `left_1013` dan `right_1013` yang digunakan sebagai penunjuk ke anak kiri dan anak kanan. Constructor pada kelas ini berfungsi untuk menginisialisasi nilai data pada node yang baru dibuat, sedangkan method setter dan getter digunakan untuk mengatur serta mengambil nilai dari masing-masing atribut.

Selain digunakan untuk menyimpan data, kelas ini juga menyediakan beberapa method traversal, yaitu `printPreorder()`, `printInorder()`, dan `printPostorder()`. Ketiga method tersebut digunakan untuk menampilkan isi pohon dengan urutan penelusuran yang berbeda. Traversal Preorder akan mencetak root terlebih

dahulu, kemudian subtree kiri dan subtree kanan. Traversal Inorder akan mencetak subtree kiri, root, lalu subtree kanan. Sedangkan Postorder akan mencetak subtree kiri, subtree kanan, kemudian root.

Pada bagian akhir terdapat method print() yang digunakan untuk menampilkan bentuk binary tree secara visual di layar.

2.3.2 Kode Kedua(BTree_2511531013)

```

1 package Pekan9_2511531013;
2 public class BTree_2511531013 {
3     private Node_2511531013 root_1013;
4     private Node_2511531013 currentNode;
5
6     public BTree_2511531013() {
7         root_1013 = null;
8     }
9
10    public boolean search(int data_1013) {
11        return search(root_1013, data_1013);
12    }
13
14    private boolean search(Node_2511531013 node_1013, int data_1013) {
15        if (node_1013.getdata_1013() == data_1013)
16            return true;
17        if (node_1013.getleft_1013() != null)
18            if (search(node_1013.getleft_1013(), data_1013))
19                return true;
20        if (node_1013.getright_1013() != null)
21            if (search(node_1013.getright_1013(), data_1013))
22                return true;
23        return false;
24    }
25
26    public void printInorder() {
27        root_1013.printInorder(root_1013);
28    }
29
30    public void printPreOrder() {
31        root_1013.printPreorder(root_1013);
32    }
33
34    public void printPostOrder() {
35        root_1013.printPostorder(root_1013);
36    }
37
38    public Node_2511531013 getRoot() {
39        return root_1013;
40    }
41    public boolean isEmpty() {
42        return root_1013 == null;
43    }
44    public int countNodes() {
45        return countNodes(root_1013);

```

```

46     }
47     private int countNodes(Node_2511531013 node_1013) {
48         int count_1013 = 1;
49         if (node_1013 == null) {
50             return 0;
51         } else {
52             count_1013 += countNodes(node_1013.getleft_1013());
53             count_1013 += countNodes(node_1013.getright_1013());
54             return count_1013;
55         }
56     }
57     public void print() {
58         root_1013.print();
59     }
60     public Node_2511531013 getCurrent() {
61         return currentNode;
62     }
63     public void setCurrent(Node_2511531013 node_1013) {
64         this.currentNode = node_1013;
65     }
66     public void setRoot(Node_2511531013 root_1013) {
67         this.root_1013 = root_1013;
68     }
69 }

```

Kode 2.2

Kode di atas merupakan kelas BTree_2511531013 yang berfungsi untuk mengelola struktur binary tree. Kelas ini memiliki atribut root_1013 sebagai akar pohon dan currentNode sebagai node yang sedang aktif. Selain itu, kelas ini menyediakan berbagai method yang digunakan untuk melakukan operasi pada binary tree, seperti pencarian data, menghitung jumlah simpul, serta menampilkan isi pohon.

Method search() digunakan untuk mencari data tertentu pada binary tree secara rekursif. Program juga menyediakan method countNodes() yang berfungsi untuk menghitung jumlah seluruh simpul pada pohon. Selain itu, terdapat method printInorder(), printPreOrder(), dan printPostOrder() yang digunakan untuk menjalankan proses traversal berdasarkan urutan yang dipilih. Method isEmpty() digunakan untuk memeriksa apakah binary tree masih kosong atau sudah memiliki data.

Pada bagian akhir terdapat beberapa method pendukung seperti setRoot(), getRoot(), setCurrent(), getCurrent(), dan print(). Method-method tersebut digunakan untuk mengatur akar pohon,

menentukan node yang sedang aktif, serta menampilkan struktur binary tree secara keseluruhan sehingga memudahkan pengguna dalam memahami bentuk pohon yang telah dibuat.

2.3.3 Kode Ketiga(BtreeDriver_2511531013)

```

1 package Pekan9_2511531013;
2 public class BtreeDriver_2511531013 {
3     public static void main(String[] args) {
4         //Membuat Pohon
5         BTree_2511531013 tree_1013 = new BTree_2511531013();
6         System.out.print("Jumlah Simpul awal pohon: ");
7         System.out.println(tree_1013.countNodes());
8         //menambahkan simpul data 1
9         Node_2511531013 root_1013 = new Node_2511531013(1);
10        //meniadikan simpul 1 sebagai root
11        tree_1013.setRoot(root_1013);
12        System.out.println("Jumlah simpul jika hanya ada root");
13        System.out.println(tree_1013.countNodes());
14        Node_2511531013 node2_1013 = new Node_2511531013(2);
15        Node_2511531013 node3_1013 = new Node_2511531013(3);
16        Node_2511531013 node4_1013 = new Node_2511531013(4);
17        Node_2511531013 node5_1013 = new Node_2511531013(5);
18        Node_2511531013 node6_1013 = new Node_2511531013(6);
19        Node_2511531013 node7_1013 = new Node_2511531013(7);
20        Node_2511531013 node8_1013 = new Node_2511531013(8);
21        Node_2511531013 node9_1013 = new Node_2511531013(9);
22        root_1013.setleft_1013(node2_1013);
23        node2_1013.setleft_1013(node4_1013);
24        node2_1013.setright_1013(node5_1013);
25        node4_1013.setright_1013(node8_1013);
26        root_1013.setright_1013(node3_1013);
27        node3_1013.setleft_1013(node6_1013);
28        node3_1013.setright_1013(node7_1013);
29        node6_1013.setleft_1013(node9_1013);
30        //Set root
31        tree_1013.setCurrent(tree_1013.getRoot());
32        System.out.println("menampilkan simpul terakhir: ");
33        System.out.println(tree_1013.getCurrent().getdata_1013());
34        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
35        System.out.println(tree_1013.countNodes());
36        System.out.println("InOrder: ");
37        tree_1013.printInorder();
38        System.out.println("\nPreorder: ");
39        tree_1013.printPreOrder();
40        System.out.println("\nPostorder : ");
41        tree_1013.printPostOrder();
42        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
43        tree_1013.print();
44    }
45 }
46

```

Kode 2.3

Kode di atas merupakan program utama yang digunakan untuk menguji seluruh fungsi pada kelas Node_2511531013 dan BTree_2511531013. Program diawali dengan membuat objek binary tree yang masih kosong, kemudian menampilkan jumlah

simpul awal sebelum data dimasukkan. Setelah itu dibuat beberapa objek node yang berisi data angka dan disusun menjadi sebuah struktur binary tree dengan menentukan hubungan antara parent, child kiri, dan child kanan.

Setelah seluruh node berhasil disusun, program menetapkan node pertama sebagai root menggunakan method `setRoot()`. Selanjutnya program menampilkan jumlah simpul yang telah terbentuk serta menentukan node aktif menggunakan method `setCurrent()`. Informasi mengenai node aktif dan jumlah simpul kemudian ditampilkan sebagai bagian dari proses pengujian program.

Pada bagian akhir, program melakukan proses traversal menggunakan metode Inorder, Preorder, dan Postorder untuk melihat urutan penelusuran simpul pada binary tree. Selain itu, program juga memanggil method `print()` sehingga struktur binary tree dapat ditampilkan dalam bentuk pohon.

2.3.4 Kode Keempat(GraphTraversal_2511531013)

```

1 package Pekan9_2511531013;
2 import java.util.*;
3
4 public class GraphTraversal_2511531013{
5     private Map<String, List<String>> graph = new HashMap<>();
6     // Menambahkan edge (graf tak berarah)
7     public void addEdge (String node1_1013, String node2_1013) {
8         graph.putIfAbsent (node1_1013, new ArrayList<>());
9         graph.putIfAbsent (node2_1013, new ArrayList<>());
10        graph.get (node1_1013).add(node2_1013);
11        graph.get(node2_1013).add(node1_1013);
12    }
13    // Menampilkan graf awal
14    public void printGraph() {
15        System.out.println("Graf Awal (Adjacency List:");
16        for (String node :graph.keySet()) {
17            System.out.print (node + "->");
18            List<String> neighbors = graph.get(node);
19            System.out.println(String.join(" ", neighbors));
20        }
21        System.out.println();
22    }
23    // DFS rekursif
24    public void dfs(String start) {
25        Set<String> visited = new HashSet<>();
26        System.out.println("Penelusuran DFS:");
27        dfsHelper(start, visited);
28        System.out.println();
29    }
30    private void dfsHelper(String current, Set<String> visited) {
31        if (visited.contains (current)) return;
32        visited.add(current);
33        System.out.print(current + " ");
34        for (String neighbor : graph.getOrDefault(current, new ArrayList<>())) {
35            dfsHelper(neighbor, visited);
36        }
37    }
38    public void bfs(String start) {
39        Set<String> visited = new HashSet<>();
40        Queue<String> queue = new LinkedList<>();

```

```

41
42     queue.add(start);
43     visited.add(start);
44
45     System.out.println("Penelusuran BFS:");
46
47     while (!queue.isEmpty()) {
48         String current = queue.poll();
49
50         System.out.print(current + " ");
51
52         for (String neighbor : graph.getOrDefault(current, new ArrayList<>())) {
53             if (!visited.contains(neighbor)) {
54                 queue.add(neighbor);
55                 visited.add(neighbor);
56             }
57         }
58     }
59
60     System.out.println();
61 }
62 // main
63 public static void main(String[] args) {
64
65     GraphTraversal_2511531013 graph = new GraphTraversal_2511531013();
66
67     // Membuat graf
68     graph.addEdge("A", "B");
69     graph.addEdge("A", "C");
70     graph.addEdge("B", "D");
71     graph.addEdge("B", "E");
72     //Cetak graf awal
73     System.out.println("Graf Awal Adalah: ");
74     graph.printGraph();
75     //lakukan penelusuran
76     graph.dfs("A");
77     graph.bfs("A");
78 }
79 }

```

Kode 2.4

Kode di atas merupakan program yang digunakan untuk mengimplementasikan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) pada sebuah graf. Graf direpresentasikan menggunakan struktur data HashMap yang berisi pasangan simpul dan daftar tetangganya (*adjacency list*). Method `addEdge()` digunakan untuk menambahkan hubungan antar simpul sehingga terbentuk sebuah graf tak berarah.

Program menyediakan dua algoritma penelusuran, yaitu DFS dan BFS. Method `dfs()` bersama `dfsHelper()` melakukan penelusuran secara mendalam dengan memanfaatkan rekursi dan HashSet untuk menyimpan simpul yang telah dikunjungi sehingga tidak terjadi pengulangan. Sementara itu, method `bfs()` melakukan penelusuran secara melebar menggunakan struktur data Queue,

sehingga seluruh simpul pada satu tingkat akan dikunjungi terlebih dahulu sebelum berpindah ke tingkat berikutnya.

Pada method `main()`, program membuat sebuah graf sederhana yang terdiri atas beberapa simpul seperti A, B, C, D, dan E. Setelah hubungan antar simpul ditambahkan menggunakan method `addEdge()`, program menampilkan bentuk graf melalui adjacency list menggunakan method `printGraph()`.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma Breadth First Search (BFS) dan Depth First Search (DFS) merupakan dua metode penelusuran graf yang memiliki cara kerja berbeda. BFS melakukan penelusuran secara melebar dengan mengunjungi seluruh simpul pada satu tingkat terlebih dahulu, sedangkan DFS melakukan penelusuran secara mendalam hingga mencapai simpul terakhir sebelum kembali ke simpul sebelumnya. Selain itu, pada praktikum ini juga dipelajari implementasi struktur binary tree beserta proses traversal menggunakan metode Preorder, Inorder, dan Postorder.

Melalui implementasi program menggunakan bahasa Java, penulis dapat memahami bagaimana membangun struktur binary tree, melakukan penelusuran pada pohon maupun graf, serta mengetahui perbedaan hasil yang diperoleh dari masing-masing algoritma. Praktikum ini memberikan pemahaman bahwa pemilihan algoritma penelusuran harus disesuaikan dengan kebutuhan dan karakteristik permasalahan yang akan diselesaikan.

3.2 Saran

Sebaiknya praktikum selanjutnya dilengkapi dengan lebih banyak contoh kasus dan variasi bentuk graf maupun binary tree sehingga mahasiswa dapat lebih memahami proses kerja setiap algoritma. Selain itu, akan lebih baik jika setiap tahap implementasi disertai penjelasan mengenai alur program dan proses traversal agar lebih mudah dipahami, terutama pada bagian yang menggunakan rekursi. Dengan demikian, mahasiswa tidak hanya mampu menjalankan program, tetapi juga memahami konsep dan logika yang digunakan dalam algoritma tersebut.

DAFTAR PUSTAKA

- [1] Trivusi, “Perbedaan DFS dan BFS,” [Daring]. Tersedia pada: <https://www.trivusi.web.id/2023/01/perbedaan-dfs-dan-bfs.html> [Diakses: 12-Jun-2026].

LAPORAN TUGAS PEKAN 8

STRUKTUR DATA

DISUSUN OLEH:

NOFRI ILHAM

2511531013

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. PetaKampus_2511531013

```
1 package Pekan9_2511531013;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.*;
6
7 public class PetaKampus_2511531013 extends JFrame {
8
9     private final Map<String, java.util.List<String>> graph_1013 = new LinkedHashMap<>();
10    private final Map<String, Point> posisi_1013 = new HashMap<>();
11
12    private final Set<String> visited_1013 = new LinkedHashSet<>();
13    private final java.util.List<String> path_1013 = new ArrayList<>();
14
15    private JComboBox<String> startBox_1013;
16    private JComboBox<String> goalBox_1013;
17    private JTextArea hasil_1013;
18    private JPanel panel_1013;
19
20    public PetaKampus_2511531013() {
21        setTitle("Peta Kampus BFS DFS - 2511531013");
22        setSize(900,650);
23        setDefaultCloseOperation(EXIT_ON_CLOSE);
24        setLocationRelativeTo(null);
25
26        inisialisasiGraph_1013();
27
28        startBox_1013 = new JComboBox<>(graph_1013.keySet().toArray(new String[0]));
29        goalBox_1013 = new JComboBox<>(graph_1013.keySet().toArray(new String[0]));
30        startBox_1013.setFont(new Font("Times New Roman", Font.PLAIN, 14));
31        goalBox_1013.setFont(new Font("Times New Roman", Font.PLAIN, 14));
32
33        JButton bfs = new JButton("BFS");
34        JButton dfs = new JButton("DFS");
35        JButton reset = new JButton("RESET");
36        Font fontButton = new Font("Times New Roman", Font.BOLD, 14);
37        bfs.setFont(fontButton);
38        dfs.setFont(fontButton);
39        reset.setFont(fontButton);
40
41        JPanel atas = new JPanel();
42        JLabel lblAwal = new JLabel("Lokasi Awal");
43        JLabel lblTujuan = new JLabel("Lokasi Tujuan");
```

```

44
45     lblAwal.setFont(new Font("Times New Roman", Font.BOLD, 14));
46     lblTujuan.setFont(new Font("Times New Roman", Font.BOLD, 14));
47
48     atas.add(lblAwal);
49     atas.add(startBox_1013);
50     atas.add(lblTujuan);
51     atas.add(goalBox_1013);
52     atas.add(bfs);
53     atas.add(dfs);
54     atas.add(reset);
55
56     hasil_1013 = new JTextArea(8,40);
57     hasil_1013.setFont(new Font("Times New Roman", Font.PLAIN, 15));
58     hasil_1013.setEditable(false);
59
60     panel_1013 = new JPanel(){
61         protected void paintComponent(Graphics g){
62             super.paintComponent(g);
63             for(String a:graph_1013.keySet()){
64                 Point p1=posisi_1013.get(a);
65                 for(String b:graph_1013.get(a)){
66                     Point p2=posisi_1013.get(b);
67                     g.setColor(Color.GRAY);
68                     g.drawLine(p1.x,p1.y,p2.x,p2.y);
69                 }
70             }
71             for(String n:graph_1013.keySet()){
72                 Point p=posisi_1013.get(n);
73                 g.setColor(path_1013.contains(n)?Color.ORANGE:visited_1013.contains(n)?Color.GREEN:Color.CYAN);
74                 g.fillOval(p.x-20,p.y-20,40,40);
75                 g.setColor(Color.BLACK);
76                 g.setFont(new Font("Times New Roman", Font.BOLD, 12));
77                 g.drawOval(p.x-20,p.y-20,40,40);
78                 g.drawString(n,p.x-20,p.y+35);
79             }
80         }
81     };
82
83     add(atas, BorderLayout.NORTH);
84     add(panel_1013, BorderLayout.CENTER);
85     add(new JScrollPane(hasil_1013), BorderLayout.SOUTH);
86
87     bfs.addActionListener(e->caribFS_1013());
88     dfs.addActionListener(e->caridFS_1013());
89     reset.addActionListener(e->resetGraph_1013());
90 }
91
92 private void tambahEdge(String a,String b){
93     graph_1013.get(a).add(b);
94     graph_1013.get(b).add(a);
95 }
96
97 private void inisialisasiGraph_1013(){
98     String[] node={"Gerbang","Parkiran","Rektorat","Perpus","Fakultas Teknik","Fakultas Kedokteran","Masjid","Kantin","Labor","Lapangan"};
99     int[][] pos={{60,250},{120,420},{220,150},{350,90},{550,90},{350,250},{220,520},{500,480},{700,180},{700,400}};
100     for(int i=0;i<node.length;i++){
101         graph_1013.put(node[i],new ArrayList<>());
102         posisi_1013.put(node[i],new Point(pos[i][0],pos[i][1]));
103     }
104     tambahEdge("Gerbang","Parkiran");
105     tambahEdge("Gerbang","Rektorat");
106     tambahEdge("Parkiran","Masjid");
107     tambahEdge("Parkiran","Lapangan");
108     tambahEdge("Rektorat","Perpus");
109     tambahEdge("Rektorat","Fakultas Kedokteran");
110     tambahEdge("Perpus","Fakultas Teknik");
111     tambahEdge("Perpus","Labor");
112     tambahEdge("Fakultas Teknik","Labor");
113     tambahEdge("Fakultas Teknik","Kantin");
114     tambahEdge("Fakultas Kedokteran","Masjid");
115     tambahEdge("Fakultas Kedokteran","Kantin");
116     tambahEdge("Masjid","Kantin");
117     tambahEdge("Kantin","Lapangan");
118     tambahEdge("Labor","Lapangan");
119 }
120
121 private void caribFS_1013(){
122     proses(true);
123 }
124 private void caridFS_1013(){
125     proses(false);
126 }
127
128 private void proses(boolean bfs){

```

```

129     visited_1013.clear();
130     path_1013.clear();
131     String start=(String)startBox_1013.getSelectedItemAt();
132     String goal=(String)goalBox_1013.getSelectedItemAt();
133     Map<String,String> parent=new HashMap<>();
134     if(bfs){
135         Queue<String> q=new LinkedList<>();
136         q.add(start);visited_1013.add(start);parent.put(start,null);
137         while(!q.isEmpty()){
138             String c=q.poll();
139             if(c.equals(goal))break;
140             for(String n:graph_1013.get(c))
141                 if(!visited_1013.contains(n)){
142                     visited_1013.add(n);parent.put(n,c);q.add(n);
143                 }
144         }
145     }else{
146         Stack<String> s=new Stack<>();
147         s.push(start);parent.put(start,null);
148         while(!s.isEmpty()){
149             String c=s.pop();
150             if(visited_1013.contains(c))continue;
151             visited_1013.add(c);
152             if(c.equals(goal))break;
153             java.util.List<String> tet=graph_1013.get(c);
154             for(int i=tet.size()-1;i>=0;i--){
155                 String n=tet.get(i);
156                 if(!visited_1013.contains(n)){
157                     if(!parent.containsKey(n))parent.put(n,c);
158                     s.push(n);
159                 }
160             }
161         }
162     }
163     if(parent.containsKey(goal)){
164         String x=goal;
165         while(x!=null){
166             path_1013.add(0,x);
167             x=parent.get(x);
168         }
169     }
170     hasil_1013.setText((bfs?"BFS":"DFS")+"\nJalur : "+String.join(" -> ",path_1013)+
171         "\nNode dikunjungi : "+visited_1013+
172         "\nJumlah node : "+visited_1013.size());
173     panel_1013.repaint();
174 }
175
176 private void resetGraph_1013(){
177     visited_1013.clear();
178     path_1013.clear();
179     hasil_1013.setText("");
180     panel_1013.repaint();
181 }
182
183 public static void main(String[] args){
184     SwingUtilities.invokeLater(()->new PetaKampus_2511531013().setVisible(true));
185 }
186 }
187

```

1. Constructor PetaKampus_2511531013

Method ini merupakan konstruktor yang akan dijalankan pertama kali saat program dibuka. Konstruktor berfungsi mengatur tampilan awal aplikasi, seperti ukuran jendela, judul program, serta memanggil method buatGraph_1013() untuk membentuk graph kampus. Selain itu, konstruktor

juga membuat seluruh komponen GUI seperti ComboBox, tombol BFS, DFS, Reset, area hasil, dan panel visualisasi graph, kemudian menghubungkan tombol dengan method yang sesuai.

2. buatGraph_1013

Method ini digunakan untuk membentuk graph yang merepresentasikan peta kampus. Di dalamnya dibuat daftar lokasi (vertex), hubungan antar lokasi (edge), serta posisi masing-masing node pada panel agar dapat ditampilkan secara visual. Method ini menjadi dasar data yang akan digunakan oleh algoritma BFS dan DFS.

3. tambahEdge_1013(String a, String b)

Method ini berfungsi menambahkan hubungan antara dua lokasi pada graph. Karena graph yang digunakan bersifat tidak berarah (undirected graph), maka setiap sisi akan ditambahkan ke kedua node sehingga keduanya saling terhubung.

4. BFS_1013

Method ini menjalankan algoritma Breadth First Search (BFS). Proses pencarian dimulai dari lokasi awal menggunakan struktur data Queue sehingga node dikunjungi berdasarkan urutan kedekatan atau level. Selama proses berlangsung, node yang telah dikunjungi disimpan agar tidak dikunjungi kembali, kemudian jalur menuju tujuan akan dibentuk menggunakan data parent setiap node.

5. DFS_1013

Method ini menjalankan algoritma Depth First Search (DFS). Berbeda dengan BFS, DFS menggunakan struktur data Stack sehingga pencarian dilakukan dengan menelusuri satu jalur sedalam mungkin sebelum kembali ke percabangan sebelumnya. Jalur yang ditemukan kemudian disimpan untuk ditampilkan pada hasil pencarian.

6. buatPath_1013

Method ini digunakan untuk menyusun jalur dari lokasi awal menuju lokasi tujuan setelah proses BFS atau DFS selesai. Jalur dibentuk menggunakan data parent dari setiap node, kemudian dibalik sehingga urutannya dimulai dari titik awal hingga titik tujuan.

7. displayPath_1013

Method ini berfungsi menampilkan hasil pencarian pada JTextArea. Informasi yang ditampilkan meliputi algoritma yang digunakan, jalur yang ditemukan, daftar node yang telah dikunjungi, serta jumlah node yang berhasil dieksplorasi selama proses pencarian.

8. displayGraph_1013

Method ini digunakan untuk memperbarui tampilan graph pada panel GUI. Setelah algoritma selesai dijalankan, method ini memanggil repaint() sehingga perubahan warna node maupun jalur dapat langsung terlihat pada visualisasi.

9. resetGraph_1013

Method ini berfungsi mengembalikan kondisi program ke keadaan awal. Seluruh data hasil pencarian seperti node yang dikunjungi, jalur, dan informasi pada area hasil akan dihapus sehingga pengguna dapat melakukan pencarian kembali dari awal.

10. resetData_1013

Method ini digunakan untuk mengosongkan data sementara yang digunakan selama proses pencarian, seperti daftar node yang telah dikunjungi dan jalur hasil pencarian. Method ini dipanggil sebelum algoritma BFS maupun DFS dijalankan agar proses pencarian dimulai dari kondisi yang bersih.

11. GraphPanel_1013.paintComponent(Graphics g)

Method ini bertugas menggambar seluruh isi graph pada panel GUI. Pertama method menggambar seluruh edge yang menghubungkan node, kemudian menggambar node sesuai posisi yang telah ditentukan. Warna node juga akan berubah sesuai kondisi, misalnya node yang telah dikunjungi akan berwarna hijau, jalur hasil pencarian berwarna oranye, sedangkan node yang belum dikunjungi menggunakan warna default.

12. main(String[] args)

Method main merupakan titik awal program dijalankan. Method ini memanggil `SwingUtilities.invokeLater()` untuk membuat objek `PetaKampus_2511531013` sehingga seluruh tampilan GUI dapat muncul dan siap digunakan oleh pengguna.

2. Output



